UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA

Ingeniería en ciencias y sistemas

Manual técnico del proyecto 2

Laboratorio de Introducción a la Programación y Computación 1

Auxiliar: José Leonel López Ajvix

Estudiante: Oliver Jorge Raxtún Morales

Carné: 202400634

Introducción

Este manual técnico describe la arquitectura, diseño e implementación del sistema Sancarlista Academy, desarrollado como Proyecto 2 del curso Introducción a la Programación y Computación 1 de la Universidad San Carlos de Guatemala.

El sistema es una aplicación de escritorio desarrollada en Java utilizando la biblioteca gráfica Swing. Implementa el patrón de diseño Modelo-Vista-Controlador (MVC) y aplica los principios fundamentales de la Programación Orientada a Objetos: herencia, polimorfismo, encapsulamiento e interfaces.

Arquitectura del Sistema

Patrón MVC

El sistema está organizado en tres capas principales siguiendo el patrón Modelo-Vista-Controlador:

Capa	Paquete	Responsabilidad
Modelo	modelo/	Entidades de datos y lógica de negocio (Usuario, Curso, Nota, etc.)
Vista	vista/	Interfaces gráficas Swing (LoginFrame, AdminFrame, etc.)
Controlador	controlador/	Orquesta la comunicación entre Modelo y Vista
Utilidades	util/	Serialización, bitácora, exportación CSV/PDF

Estructura de Paquetes

```
src/
├── Main.java                    ← Punto de entrada
├── modelo/                      ← Entidades del dominio
│   ├── Usuario.java             ← Clase abstracta base
│   ├── Administrador.java
│   ├── Instructor.java
│   ├── Estudiante.java
│   ├── Curso.java
│   ├── Nota.java
│   ├── EntradaBitacora.java
│   └── SistemaDatos.java        ← Almacén central serializable
├── vista/                       ← Interfaces gráficas
│   ├── LoginFrame.java
│   ├── AdminFrame.java
│   ├── InstructorFrame.java
│   ├── EstudianteFrame.java
│   └── ReportesFrame.java
├── controlador/
│   └── SistemaController.java
└── util/
    ├── Bitacora.java
    ├── Serializacion.java
    ├── ExportadorCSV.java
    └── ExportadorPDF.java
```

Descripción de Clases

Jerarquía de Usuario (Herencia)

La clase Usuario es abstracta y sirve como base para los tres roles del sistema. Esto implementa el principio de herencia de la POO.

Clase	Hereda de	Descripción
Usuario	—	Clase abstracta. Atributos: codigo, nombre, fechaNacimiento, genero, contrasena, rol. Método abstracto: getDescripcion()
Administrador	Usuario	Rol con acceso total. Gestiona instructores, estudiantes, cursos y secciones.
Instructor	Usuario	Gestiona secciones asignadas. Contiene arreglo de secciones. Método: tieneSeccion(), agregarSeccion().
Estudiante	Usuario	Se inscribe en secciones. Contiene arreglo de inscripciones. Método: estaInscrito(), inscribirSeccion().

Clase SistemaDatos

Almacén central de datos. Implementa Serializable para persistencia. Usa arreglos nativos Java (sin ArrayList, HashMap u otras colecciones prohibidas).

Método principal	Retorno	Descripción
agregarUsuario(Usuario)	boolean	Agrega usuario validando código único
buscarUsuario(String)	Usuario	Busca por código, retorna null si no existe
eliminarUsuario(String)	boolean	Elimina con desplazamiento del arreglo
getUsuariosPorRol(String)	Usuario[]	Filtra usuarios por rol (ADMIN/INSTRUCTOR/ESTUDIANTE)
agregarCurso(Curso)	boolean	Valida código único antes de insertar
agregarNota(Nota)	boolean	Registra calificación en el arreglo de notas
calcularPromedio(String,String)	double	Promedio ponderado: Σ(nota×pond)/Σ(pond)
registrarEvento(EntradaBitacora)	void	Agrega entrada a la bitácora del sistema

Clase SistemaController

Controlador principal (MVC). Recibe peticiones de la Vista, aplica lógica de negocio y actualiza el Modelo. Gestiona también la persistencia automática.

Método	Descripción
login(codigo, contrasena)	Autentica usuario, incrementa sesiones activas, registra en bitácora
logout()	Cierra sesión, decrementa contador, registra evento

crearInstructor(...)	Valida unicidad, crea objeto Instructor, serializa
crearEstudiante(...)	Valida unicidad, crea objeto Estudiante, serializa
crearCurso(...)	Valida código único y créditos enteros, serializa
crearNota(...)	Valida instructor-sección, estudiante inscrito, rango 0-100
inscribirEstudiante(...)	Verifica no duplicado, llama est.inscribirSeccion()
desasignarEstudiante(...)	Bloquea si hay notas registradas en la sección
cargarInstructoresCSV(...)	Parsea CSV línea por línea, reporta ok/errores

Clase Nota

Representa una calificación registrada por un instructor para un estudiante en una sección específica.

- codigoCurso: identificador del curso
- codigoSeccion: identificador de la sección
- codigoEstudiante: identificador del estudiante calificado
- etiqueta: nombre de la actividad (ej. "Parcial 1", "Tarea 3")
- ponderacion: peso en porcentaje (debe ser > 0)
- nota: calificación de 0 a 100
- fechaRegistro: formato YYYY-MM-DD

Clase EntradaBitacora

Registra cada evento del sistema con el formato:

```
[DD/MM/YYYY HH:mm:ss] | [USUARIO_TIPO] | [CODIGO] | [OPERACION] | [ESTADO] |
[DESCRIPCION]
```

Persistencia de Datos

Serialización Java

El sistema utiliza serialización nativa de Java para persistir el estado completo entre sesiones. La clase SistemaDatos implementa la interfaz Serializable y se almacena en el archivo datos_sistema.ser.

- Al iniciar: Serializacion.cargar() lee el archivo .ser o crea SistemaDatos nuevo
- Al guardar: Serializacion.guardar() se llama automáticamente después de cada operación de escritura
- Recuperación: si el archivo está corrupto o no existe, el sistema inicia en estado limpio

Nota sobre el archivo .ser

IMPORTANTE: Si se modifica la contraseña del administrador en el código fuente, es necesario eliminar el archivo datos_sistema.ser para que el sistema cargue la nueva configuración, ya que el archivo serializado tiene prioridad sobre el código.

Procesamiento con Hilos

El sistema implementa tres hilos de fondo (daemon threads) que se ejecutan en paralelo sin bloquear la interfaz gráfica:

Hilo	Intervalo	Función
Thread-Sesiones	10 segundos	Muestra usuarios activos y timestamp de última actividad
Thread-Inscripciones	8 segundos	Contador de inscripciones pendientes en cola
Thread-Estadísticas	15 segundos	Cursos activos, estudiantes registrados, notas totales

Implementación técnica:

- Se usa Thread con método run() que contiene un bucle while(hilosActivos)
- Thread.sleep() controla el intervalo de actualización
- SwingUtilities.invokeLater() actualiza la UI de forma segura desde el hilo secundario
- La bandera booleana volatile hilosActivos se establece en false al cerrar sesión
- Los hilos son daemon (setDaemon(true)) para terminar automáticamente al cerrar la JVM

Formatos de Importación CSV

Instructores

```
Codigo,Nombre,FechaNacimiento,Genero,Contrasena
INS001,Juan Pérez,1985-03-15,M,pass123
```

Estudiantes

```
Codigo,Nombre,FechaNacimiento,Genero,Contrasena
EST001,María García,2000-07-22,F,est456
```

Cursos

```
Codigo,NombreCurso,Descripcion,Creditos,Seccion
CUR001,Matemáticas I,Cálculo diferencial,4,SEC-A
```

Notas

```
CodigoCurso,CodigoSeccion,CodigoEstudiante,Ponderacion,Nota,Fecha
CUR001,SEC-A,EST001,30,85,2025-10-15
```

Módulo de Reportes

Los reportes se generan mediante la clase ExportadorPDF (texto formateado) y ExportadorCSV. La nomenclatura de archivos sigue el formato:

```
DD_MM_YYYY_HH_mm_ss_TipoReporte.txt / .csv
```

Reporte	Contenido
Mejor Desempeño	Top 5 estudiantes con mayor promedio ponderado en una sección
Bajo Desempeño	Top 5 con menor promedio + recomendaciones académicas
Calificaciones por Sección	Lista completa de notas con aprobados/reprobados
Historial Individual	Todas las secciones de un estudiante con promedios y estados
Inscripciones por Curso	Cantidad de inscritos por sección del curso

Seguridad y Validaciones

- Sistema cerrado: solo usuarios registrados por el Administrador pueden acceder
- Las contraseñas se almacenan en texto plano en el .ser (mejora futura: hashear con SHA-256)
- Control de acceso por rol: cada Frame verifica que el usuario activo tenga el rol correcto
- No se permite desasignar estudiantes con notas registradas
- Las notas deben estar en rango 0-100 y la ponderación debe ser > 0
- Códigos únicos validados en instructores, estudiantes y cursos
- Bitácora registra todos los eventos incluyendo intentos fallidos de login

Requisitos del Sistema

Requisito	Especificación
Lenguaje	Java SE 8 o superior
IDE recomendado	NetBeans 12+ (incluye GUI Builder Swing)
JDK mínimo	JDK 8 (Java 8)
Memoria RAM	Mínimo 256 MB disponibles
Espacio en disco	Mínimo 10 MB para el proyecto + datos
Sistema Operativo	Windows, Linux o macOS con JRE instalado
Librerías externas	Ninguna (solo Java SE estándar)